

RAG System

Architectural Design and Implementation

Giovanni De Francesco

Università degli Studi di Napoli Federico II

Text Mining Exam

September 18, 2026

Contents

1	Introduction	3
1.1	Context	3
1.1.1	Dataset Structure	3
1.1.2	Document Format and Metadata	4
2	RAG Implementations without LangChain	5
2.1	Shared Document Ingestion Pipeline	5
2.1.1	Document Loading and Metadata Preparation	5
2.1.2	Chunking and Embedding Generation	6
2.1.3	Vector Storage and Retrieval Integration	6
2.2	Single-Agent Workflow	6
2.2.1	Input Processing and Routing	8
2.2.2	Country Identification and Search Scope	8
2.2.3	Retrieval and Reranking	8
2.2.4	Answer Generation and Verification	8
2.2.5	Storage and Response Delivery	9
2.3	Single-Agent Architecture and Main Modules	9
2.3.1	Configuration Management	9
2.3.2	Document Retrieval and Vector Stores	9
2.3.3	Answer Coordination	10
2.3.4	Memory and Answer Verification	10
2.3.5	Interfaces, Logging, and Persistence	10
2.4	Multi-Agent Workflow	11
2.4.1	Specialist Descriptions and Selection	11
2.4.2	Query Dispatch and Knowledge-Base Routing	11
2.4.3	Construction of Specialist Contexts	13
2.4.4	Collection and Synthesis of Partial Answers	13
2.4.5	Citation Verification and Final Response	13
2.5	Multi-Agent Architecture and Main Modules	14
2.5.1	Configuration Management	14
2.5.2	Query Rewriting and Routing	14

2.5.3	Answer Generation	14
2.5.4	Verification and Interaction Management	15
3	LangChain RAG Implementation	16
3.1	LangChain single Agent Workflow	17
3.2	Single-Agent Workflow	17
3.3	Overall Architecture	19
3.4	LangChain Multi-Agent Workflow	19
3.5	Overall Architecture	21
3.5.1	Infrastructure and Specialist Registry	21
3.5.2	Orchestration and Pipeline Routing	21
3.5.3	Specialist Retrieval and Answer Generation	21
3.5.4	Synthesis, Validation, and Memory	22
4	Evaluation Metrics and Conclusion	23
4.1	Evaluation Methodology	23
4.2	Evaluation Metrics	23
4.3	Influence of Reference-Answer Quality	25
4.4	Aggregation and Interpretation	25
4.5	Ragas Results	26
4.5.1	Performance Comparison	26

Chapter 1

Introduction

This report presents the design and implementation of a Retrieval-Augmented Generation (RAG) system, outlining the project objectives, methodology, experimental results, and main conclusions.

The study investigates two RAG architectures:

- a single-agent architecture;
- a multi-agent architecture.

Each architecture is evaluated in two configurations:

1. **Without LangChain:** the system is implemented without the LangChain framework and uses a specific embedding model.
2. **With LangChain:** the system is implemented using the LangChain framework and a different embedding model.

1.1 Context

Both architectures use a reference dataset called *Context Data*, which consists of three country-specific subsets: Italy, Estonia, and Slovenia.

1.1.1 Dataset Structure

For each country, the corpus is organized into three main content categories:

- **Inheritance law:** civil code provisions governing inheritance and succession.
- **Divorce and family law:** civil code provisions and related rules governing separation, divorce, and family matters.
- **Past legal cases:** case materials concerning inheritance and divorce, including court decisions, settlements, and mediation outcomes.

1.1.2 Document Format and Metadata

All documents are stored in JSON format. Each file contains a text field holding the content of a legal provision or case, together with associated metadata.

The metadata typically include the following:

- **Country:** Italy, Estonia, or Slovenia.
- **Legal area:** inheritance or divorce.
- **Referenced provisions:** relevant civil code articles, where applicable.
- **Document type:** `code` for legal provisions or `case` for case materials.
- **Source:** the source from which the document was obtained.

Additional metadata may describe the duration and cost of proceedings, the nature of the dispute, and other contextual information.

Chapter 2

RAG Implementations without LangChain

This chapter examines single-agent and multi-agent Retrieval-Augmented Generation (RAG) implementations developed without LangChain. Both approaches use the same legal document collection, ingestion pipeline, embedding model, and language model configuration. This shared foundation supports a comparison of how architectural differences affect retrieval and answer generation.

Both documents and queries are embedded using `BAAI/bge-m3`, which produces 1,024-dimensional vectors. The following section describes the shared ingestion process.

2.1 Shared Document Ingestion Pipeline

The ingestion pipeline converts legal documents into a searchable knowledge base used by both implementations. Defined in `Ingestion.ipynb`, it consists of five main stages: document validation, metadata preparation, chunking, embedding generation, and storage in Pinecone.

2.1.1 Document Loading and Metadata Preparation

The dataset consists of JSON files organized by country and document category. Each file contains document text and associated metadata. The pipeline checks the directory structure, parses each file, and verifies that the document text is non-empty. Loading errors and missing legal-area values cause processing to stop before uploading begins.

Each document is assigned metadata identifying its country, legal area, document type, and source path. Document types are classified as `Legal Cases` or `Civil Codes`. For legislation, the legal area is derived from the folder name; for judicial decisions, it is read from the original metadata. Legal-area values in case metadata must therefore match the vocabulary used by the retrieval filters.

Citation labels combine the country and legal area with a case identifier or legislative reference. The filename is used when the preferred identifier is unavailable. These labels support source attribution during answer generation and subsequent verification.

2.1.2 Chunking and Embedding Generation

Document text is divided into chunks of up to 1,500 tokens using the BGE-M3 tokenizer, with an overlap of 150 tokens between adjacent chunks. Documents shorter than this limit remain intact. The overlap helps preserve context across adjacent passages, although fixed token boundaries may split sentences or legal provisions.

BGE-M3 uses a **subword tokenizer**, which divides rare or technical terms into smaller recurring units while retaining common words as single tokens when possible.

Before embedding, each chunk is enriched with a compact header and its metadata. BGE-M3 then generates a normalized, 1,024-dimensional dense vector for each enriched chunk.

Embedding generation runs on a CUDA-enabled GPU when available and falls back to the CPU otherwise. During retrieval, the same model encodes user queries into the same vector space.

2.1.3 Vector Storage and Retrieval Integration

Vectors are uploaded to the shared **legal-rag** Pinecone index, which is configured for 1,024-dimensional vectors and cosine similarity. Each record contains an embedding, source text, metadata, a citation label, and the chunk’s position within the source document. A deterministic identifier derived from the source path and chunk position ensures that repeated uploads target the same record.

Before uploading, the notebook validates the index configuration and reports document counts. In its current configuration, it clears the target namespace and rebuilds the collection from the supplied dataset.

Both the single-agent and multi-agent implementations retrieve documents from this index using metadata filters. In the single-agent system, the router determines the search scope. In the multi-agent system, specialist agents apply filters corresponding to their respective jurisdictions, legal areas, and document types.

2.2 Single-Agent Workflow

Figure 2.1 illustrates a conversation turn, from user input to response delivery. Each subsequent message starts a new turn using the saved conversation context.

The diagram uses blue for memory operations, green for user interaction, and yellow for internal response preparation.

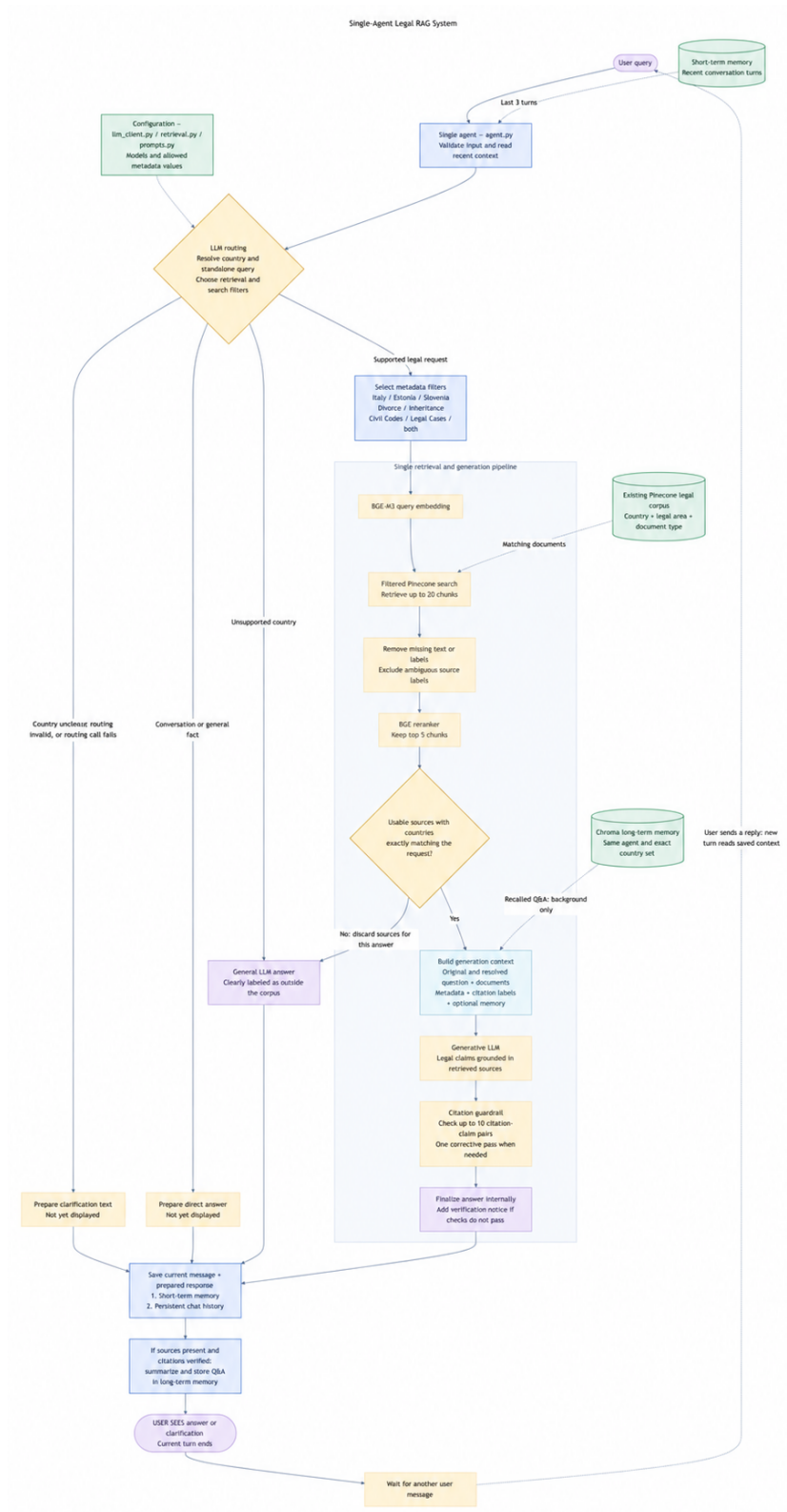


Figure 2.1: Single-agent RAG workflow.

2.2.1 Input Processing and Routing

The user submits a question through the Gradio interface. The system validates the input and reads the last three conversation turns from short-term memory to interpret follow-up questions.

A single LLM call determines whether retrieval is needed, rewrites the question as a standalone query, and selects metadata filters. The diagram separates these operations for clarity. Conversational requests and general factual questions may receive a direct answer, while jurisdiction-dependent legal questions favor retrieval. Unclear jurisdictions, invalid routing responses, or routing-call failures produce a clarification request.

2.2.2 Country Identification and Search Scope

The router identifies countries from the current question or unambiguous references in earlier user messages. The corpus covers Italy, Estonia, and Slovenia. Requests involving unsupported countries use a general-knowledge LLM fallback, explicitly identified as not grounded in the retrieved corpus.

For supported requests, the router selects the legal area (**Divorce** or **Inheritance**) and document types. Explicit case-law requests select **Legal Cases**; questions about statutory provisions, rights, obligations, or legal conditions select **Civil Codes**. Requests combining rules and judicial practice, or leaving the source type unspecified, can search both.

2.2.3 Retrieval and Reranking

BGE-M3 encodes the standalone query for a filtered Pinecone search, which returns up to 20 candidate chunks. Records without usable text or citation labels are excluded, together with labels shared by different sources.

The multilingual cross-encoder **BAAI/bge-reranker-v2-m3** scores each query–passage pair, including relevant passage metadata, and retains up to five chunks.

Every retained chunk must contain country metadata, and their combined country set must exactly match the requested set. Empty results or incomplete coverage trigger the LLM fallback. This check establishes jurisdictional coverage, but not coverage of every aspect of the question.

2.2.4 Answer Generation and Verification

When suitable sources are available, the system recalls relevant long-term summaries scoped to the same agent and country set. These summaries provide background context, while retrieved documents serve as the evidence for legal claims.

The answer model receives the original question, its standalone reformulation, recent conversation history, retrieved documents, and recalled summaries. Before the answer is returned to the user, the output guard checks up to ten citation–claim pairs per pass. If it detects citations that are absent from the retrieved sources or claims unsupported by their cited sources, it requests one corrective rewrite and checks the revised answer again. Any remaining issues or incomplete verification are reported in a notice accompanying the answer. Direct answers and fallback answers generated without suitable sources bypass this guard.

2.2.5 Storage and Response Delivery

Each completed turn saves the user message and finalized response in short-term memory and persistent chat history, including clarifications, direct answers, and fallbacks. Long-term storage is attempted only when retrieved documents are present and citation verification succeeds. The response is then returned to the interface.

For example, “**How does divorce work?**” may prompt a request to specify the country. If the user replies “**Italy**”, the next turn uses the saved context to reconstruct “**How does divorce work in Italy?**” and proceeds through retrieval and answer generation. The flowchart’s return arrow therefore represents a new user message, rather than an automatic continuation of the previous turn.

2.3 Single-Agent Architecture and Main Modules

The single-agent implementation coordinates routing, retrieval, answer generation, verification, and memory through a single application controller. Its principal modules are described below.

2.3.1 Configuration Management

The `llm_client.py` module centralizes language-model configuration. It loads environment settings and exposes `get_llm_client()` and `model_names()`, ensuring consistent configuration across answer generation, citation verification, and memory summarization.

Retrieval settings specify the Pinecone index and namespace. Supported countries, legal areas, and document types are defined in `retrieval.py`.

2.3.2 Document Retrieval and Vector Stores

The `PineconeRetriever` class in `retrieval.py` provides access to the legal corpus. The `build_metadata_filter()` function converts the router’s selections into validated metadata filters.

The retriever generates normalized BGE-M3 query embeddings, searches Pinecone, validates the returned records, and reranks the candidates using a BGE cross-encoder.

A separate Chroma database, managed by `memory/long_term.py`, stores summaries of previous question-answer exchanges. It uses `all-mpnet-base-v2` embeddings for semantic recall. Pinecone supplies documentary evidence, while Chroma supplies conversational background. The application initializes the conversational store without rebuilding the legal-document index.

2.3.3 Answer Coordination

The `SingleAgentRAG` class in `agent.py` coordinates the workflow through its `ask()` method.

The `_triage_and_route()` method determines whether retrieval is required, resolves contextual references, and selects metadata filters. Based on this decision, the system responds directly, requests clarification, or retrieves documents.

When the retrieved sources satisfy the country-coverage check, `_answer_with_rag()` generates a response grounded in those sources. Unsupported jurisdictions, empty retrieval results, or incomplete country coverage trigger `_answer_without_sources()`.

All these paths are managed by a single coordinating agent. The implementation neither delegates questions to specialist agents nor aggregates independently generated specialist answers.

2.3.4 Memory and Answer Verification

The `memory` package manages recent conversation context, long-term semantic recall, and persistent chat history as separate components. The router receives the last three turns to interpret follow-up messages.

The `guardrails/output_guard.py` module exposes `check_rag_answer()`, which checks citation grounding before a retrieval-based answer is returned to the user. If it detects unknown citations or unsupported claims, it may request one corrective rewrite, followed by a second check. Unresolved issues or incomplete verification are disclosed in a notice accompanying the response. Only retrieval-based answers that pass citation verification are eligible for long-term storage.

2.3.5 Interfaces, Logging, and Persistence

The `main.py` module provides a command-line interface, while `ui.py` provides a Gradio interface with conversation navigation. Operational logs record routing decisions, selected filters, and execution problems.

The `ChatHistoryStore` class in `memory/chat_history.py` stores completed exchanges in SQLite and maintains a JSON export. Retrieved-document information is linked to the corresponding interactions to support inspection and evaluation.

2.4 Multi-Agent Workflow

MultiAgentRag uses a supervisor-based architecture in which a central *supervisor agent* coordinates specialized legal agents. It routes the user’s request, dispatches the resolved query, and combines the specialists’ answers. This separates workflow coordination from legal evidence retrieval and analysis. Figure 2.2 illustrates the main stages.

2.4.1 Specialist Descriptions and Selection

Each specialist is defined in an agent registry by a unique identifier, a natural-language description, supported countries, legal areas, source types, and a metadata filter. These descriptions guide selection by distinguishing expertise, such as statutory provisions or judicial decisions.

The supervisor delegates query rewriting, jurisdiction resolution, and specialist selection to a routing component. In a single structured language-model call, the router uses recent conversation history to reformulate context-dependent requests as standalone questions and select relevant agents.

Country selection is validated against the user’s messages. Missing or ambiguous jurisdictions trigger clarification before retrieval, rather than an automatic search across all three supported countries: Italy, Slovenia, and Estonia.

2.4.2 Query Dispatch and Knowledge-Base Routing

The supervisor sends the resolved query to the selected specialists, with up to four running concurrently. Each receives the same question but retrieves evidence only within its assigned scope.

The knowledge bases shown in the flowchart are logical subsets of a shared Pinecone index. Routing operates at two levels: the supervisor selects specialists by expertise, and each specialist applies its registry-defined metadata filter to restrict retrieval by country, legal area, and, where applicable, source type. The registry defines agent coverage; the index stores the evidence.

Conversational requests may receive direct responses. Questions about countries outside the corpus receive separately labelled general language-model answers. Mixed requests, such as a comparison between Italy and France, combine a retrieval-backed section for Italy with a general section for France. The flowchart simplifies this behaviour by depicting the external-country response as a separate branch.

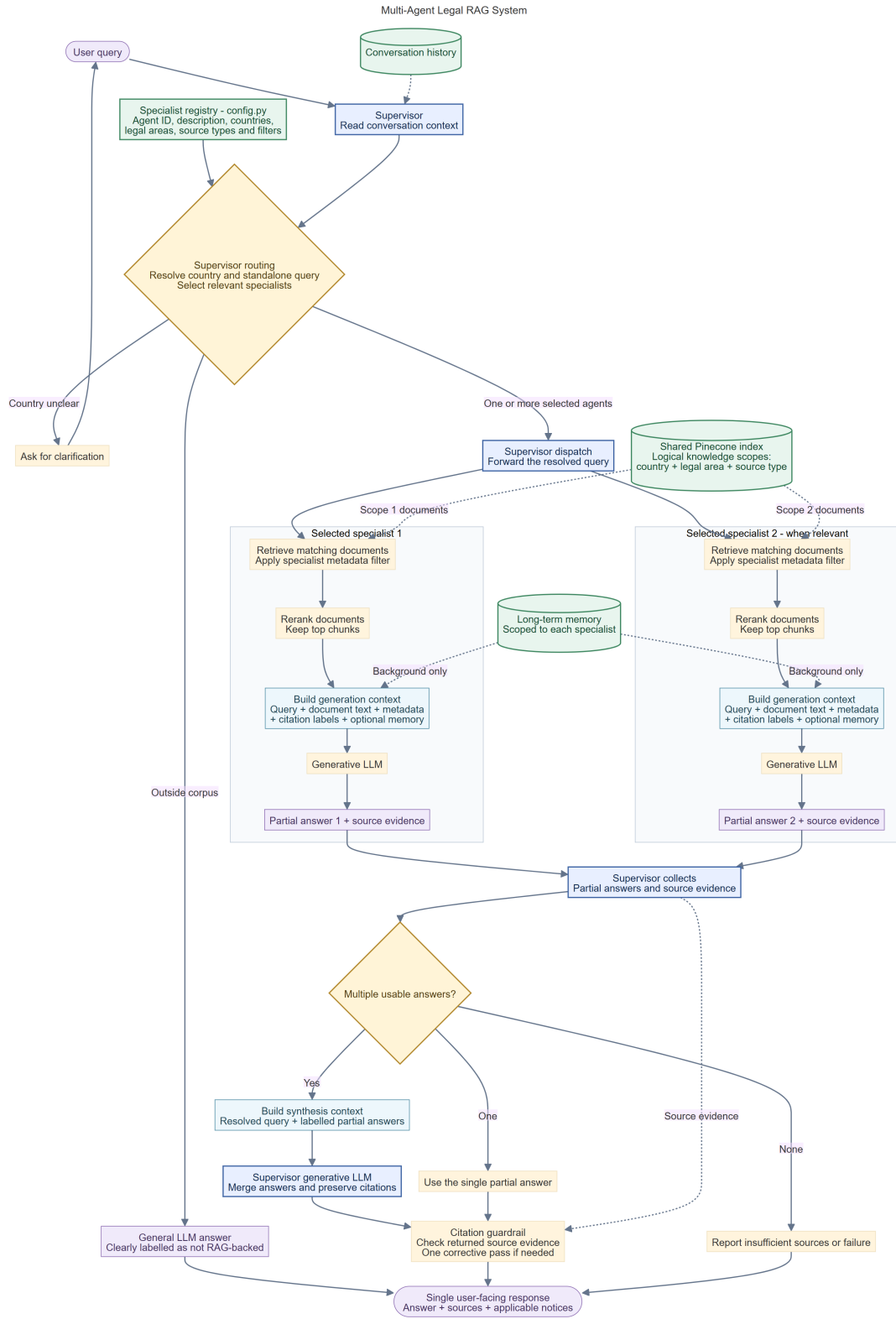


Figure 2.2: MultiAgentRag workflow.

2.4.3 Construction of Specialist Contexts

Each specialist embeds the resolved query and performs a filtered similarity search, retrieving up to twenty candidate chunks. A cross-encoder reranks them, and the top five are considered for context construction. Usable chunks are assembled with source metadata and unambiguous citation labels.

The generation context contains the resolved question and retrieved evidence. It may also include summaries of similar past interactions, recalled from long-term memory using the specialist’s identifier. These summaries provide conversational background, not documentary evidence.

The language model generates a partial answer citing the supplied sources. Each specialist returns a structured result containing its identifier, answer, labelled evidence, retrieved-document metadata, and relevance score.

2.4.4 Collection and Synthesis of Partial Answers

The supervisor collects usable specialist results. A single partial answer is used directly; multiple answers are passed to a synthesis model. If none are usable, the system reports insufficient sources or an execution failure.

The synthesis context contains the resolved query and labelled partial answers, without reassembling the retrieved chunks. The model merges the contributions while preserving citations, combining findings within each jurisdiction and organizing cross-country answers comparatively. Source evidence remains separately available for citation verification and reporting.

2.4.5 Citation Verification and Final Response

An output guardrail checks whether cited evidence supports the associated statements in the retrieval-backed answer. It checks up to ten claim–citation pairs per pass and allows one corrective rewrite followed by verification. Notices report unresolved citation issues or incomplete checks. This assesses citation support without guaranteeing the accuracy of every assertion.

The supervisor appends source information and any separately labelled external-country answer, then saves the response to conversation history and short-term memory. The retrieval-backed portion may also be summarized into long-term memory through a background task. External-country answers bypass corpus-based citation verification and are excluded from long-term storage.

2.5 Multi-Agent Architecture and Main Modules

MultiAgentRag separates configuration management, query routing, answer generation, and response verification into dedicated modules. A central supervisor coordinates specialized agents covering divorce and inheritance law in Italy, Estonia, and Slovenia. As described previously, document ingestion is shared with the single-agent implementation. The multi-agent system accesses the resulting Pinecone index through specialist-specific metadata filters.

2.5.1 Configuration Management

The `config.py` module defines the specialist registry. Each entry specifies an agent’s identifier, natural-language description, jurisdiction, legal areas, source types, and retrieval filter. Descriptions guide agent selection, while filters restrict the evidence available to each specialist.

The registry contains eleven specialists: four each for Italy and Slovenia, and three for Estonia. Legislation and case law are handled separately for each legal area, except for Estonian inheritance law, which uses a combined specialist.

The `llm_client.py` module centralizes provider configuration and model assignments for answer generation, citation verification, and memory summarization. Credentials and provider selection are loaded from environment settings, while retrieval models and concurrency parameters are defined in `agents.py`.

2.5.2 Query Rewriting and Routing

The `ask()` method of `SupervisorAgent` coordinates the online workflow. Through the `routing.py` module, it uses recent conversation context to rewrite context-dependent requests as standalone questions, resolve the jurisdiction, and select relevant specialists. These operations are performed within a single language-model call.

The routing result is validated against the user’s messages. Missing or ambiguous jurisdictions trigger clarification before retrieval, preventing the system from automatically extending an unspecified request to all supported countries.

Conversational requests can receive direct responses without retrieval. Questions concerning countries outside the corpus receive general language-model answers, explicitly labelled as not supported by retrieved sources. Mixed requests combine a corpus-based answer for supported countries with a separately labelled answer for external countries.

2.5.3 Answer Generation

The supervisor sends the same resolved query to the selected specialists, with up to four executing concurrently. Each specialist’s `answer()` method embeds the query and

retrieves up to twenty candidate chunks from Pinecone using its configured metadata filter. A cross-encoder reranks these candidates, and the top five are considered for context construction.

The generation context combines the resolved question with usable source passages, metadata, and citation labels. It may also include summaries of similar past interactions recalled from long-term memory for the selected specialist. These summaries provide conversational background and are not treated as documentary evidence. The language model produces a partial answer, returned alongside its source evidence, document metadata, and relevance score.

A single usable partial answer proceeds directly to verification. When several are available, the supervisor constructs a synthesis context from the resolved query and partial answers. The synthesis model combines their contributions while preserving citations, using a comparative structure when multiple jurisdictions are involved. Retrieved passages remain separately available for verification rather than being included again in the synthesis context. If no usable partial answers are returned, the system reports insufficient sources or an execution failure.

2.5.4 Verification and Interaction Management

The `guardrails/output_guard.py` module checks whether cited sources support their associated claims. It evaluates up to forty claim–citation pairs per pass and permits one corrective rewrite, followed by another verification pass. Unresolved citation issues and incomplete checks are reported through notices. This process assesses citation support rather than guaranteeing the accuracy of every statement.

The supervisor appends source information and any external-country answer, then saves the completed response to conversation history and short-term memory. A background process may summarize the retrieval-backed portion for long-term semantic memory. External-country answers bypass corpus-based citation verification and are excluded from long-term storage.

The `ui.py` module provides the Gradio interface for submitting questions, inspecting source information, and managing conversations.

Chapter 3

LangChain RAG Implementation

The LangChain-based single-agent and multi-agent systems share the same document ingestion pipeline and legal corpus. The dataset is unchanged from the previous implementations and covers divorce and inheritance law in Italy, Estonia, and Slovenia.

LangChain was introduced to explore how its modular interfaces affect code organization, component integration, and reuse while preserving the overall system architectures. A further objective was to investigate how a different embedding model influences retrieval quality and, consequently, the generated answers. Access to a rented GPU server on Paperspace enabled experiments with more computationally demanding embedding models. These experiments examine both implementation choices and embedding performance; changes in answer quality should therefore not be attributed to LangChain alone.

Document Preparation and Embedding Generation

The pipeline validates the input JSON documents and standardizes their metadata, including country, legal area, document type, and source. Each document receives a citation label for source attribution.

Using the tokenizer associated with NVIDIA’s `Nemotron-3-Embed-1B-BF16`, documents are divided into chunks of up to 1,500 tokens with a 150-token overlap; shorter documents remain intact. This overlap limits context loss, although chunk boundaries may still divide sentences or legal provisions.

Chunks are enriched with metadata before a shared LangChain adapter generates normalized, 2,048-dimensional embeddings on a CUDA GPU. Passages and queries use the prefixes `passage:` and `query:`, respectively, with consistent model and preprocessing settings across ingestion and retrieval.

Vector Storage and Retrieval Integration

Embeddings are stored in a shared Pinecone index using cosine similarity, alongside the original chunk text, metadata, citation label, and chunk position. Records and index

dimensions are validated before batch uploads, which include retries for transient failures. Deterministic identifiers prevent duplicate records when unchanged documents are re-ingested, while a local manifest records the ingestion configuration and document and vector counts.

The single-agent system derives retrieval filters from the query; the multi-agent system applies filters according to each specialist’s scope. Using the same corpus and vector representations allows the comparison to focus on differences in routing, retrieval, and answer generation.

3.1 LangChain single Agent Workflow

3.2 Single-Agent Workflow

Figure 3.1 illustrates the single-agent workflow: the system interprets the question, selects retrieval filters, retrieves and reranks relevant passages, and generates an answer whose citations are checked before storage and delivery. The LangChain and non-LangChain implementations share essentially the same architecture. Their differences are primarily at the implementation level: LangChain provides standardized interfaces and composable processing stages, while the embedding model also differs.

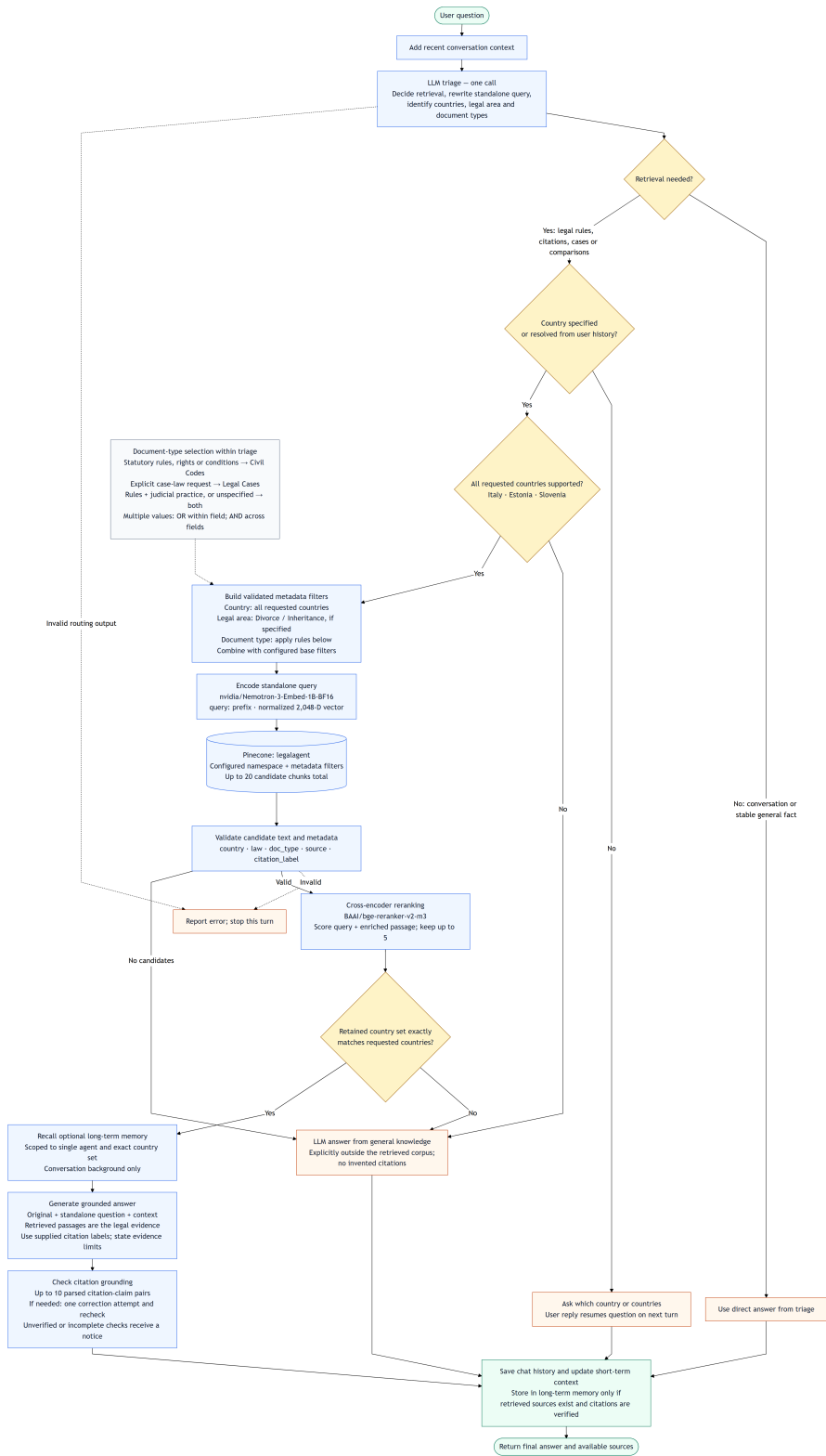


Figure 3.1: LangChain single-agent workflow.

3.3 Overall Architecture

Workflow Orchestration

The main change introduced by LangChain concerns the organization and orchestration of the code. In the original implementation, `agent.py` coordinates the processing stages through explicit procedural calls. In the LangChain version, `agent.ipynb` composes these stages using `RunnableLambda` and `RunnableBranch`. The former wraps operations such as routing, retrieval, generation, and persistence; the latter selects between alternative paths, including direct responses, retrieval-grounded answers, and general-knowledge fallback.

Model and Retrieval Interfaces

The component interfaces are also standardized. `llm_client.ipynb` initializes native LangChain chat models, allowing the agent to invoke different providers through a common interface. Similarly, `retrieval.ipynb` implements a LangChain retriever that combines query embedding, filtered Pinecone search, and reranking. Retrieved passages are returned as `Document` objects, keeping their text and metadata together as they pass through the workflow.

Citation Verification, Memory, and User Interface

The remaining notebooks preserve the original responsibilities. `output_guard.ipynb` exposes citation verification as a runnable, while `short_term.ipynb`, `long_term.ipynb`, and `chat_history.ipynb` manage recent context, persistent semantic memory, and conversation records, respectively. These modules are loaded by `agent.ipynb` and connected to the user interface in `ui.ipynb`.

Architectural Continuity and Embedding Changes

LangChain changes how the existing components are implemented and composed, rather than introducing a fundamentally different architecture. The replacement of BGE-M3 with Nemotron in the embedding stage is a separate experimental choice, intended to investigate its effect on retrieval and answer quality.

3.4 LangChain Multi-Agent Workflow

The LangChain-based system follows the same overall workflow as the previous multi-agent implementation: a supervisor selects relevant legal specialists, forwards the query, and combines their cited partial answers into a single response. The main difference lies in the implementation, which uses LangChain interfaces to integrate the components.

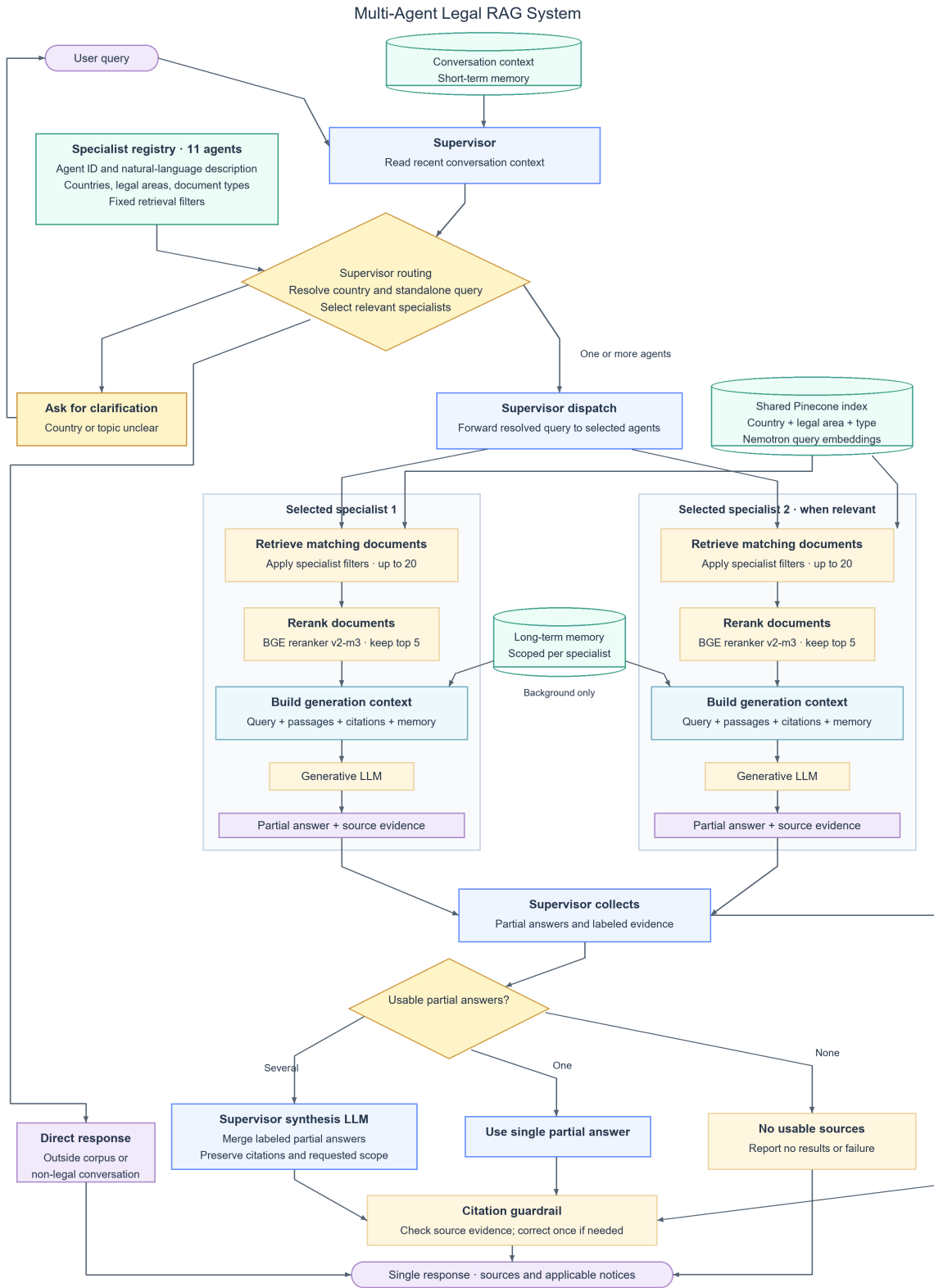


Figure 3.2: LangChain multi-agent workflow.

3.5 Overall Architecture

The system employs a modular multi-agent architecture that separates shared infrastructure, specialist knowledge, and workflow coordination.

3.5.1 Infrastructure and Specialist Registry

At the foundational level, a centralized configuration module manages API credentials, language-model selection, token limits, and connections to the embedding model, reranker, and vector database. This shared infrastructure ensures consistency across agents while allowing individual components to be updated independently. A specialist registry describes the available agents by defining their jurisdictions, legal domains, supported document types, role descriptions, and metadata filters. The registry therefore provides the supervisor with the information required to select the most appropriate specialists for each request.

3.5.2 Orchestration and Pipeline Routing

The workflow is orchestrated by the `SupervisorAgent.ask()` method, which acts as the main entry point to the system. It receives the user’s message, loads the relevant conversation history, and coordinates the answer-generation process. Its first stage, `_triage_and_route()`, analyses the current request together with the preceding dialogue and selects one of three pipelines.

The direct-answer pipeline handles conversational or general requests that do not require documentary evidence, thereby avoiding unnecessary retrieval. The clarification pipeline is activated when the request is ambiguous or lacks information required for reliable routing. In this case, the supervisor asks a focused follow-up question instead of selecting a specialist based on incomplete information. The user’s reply is subsequently combined with the stored history to construct a precise, self-contained query.

3.5.3 Specialist Retrieval and Answer Generation

For domain-specific questions requiring supporting evidence, the supervisor activates the specialist retrieval pipeline. After identifying the relevant legal areas or jurisdictions, it invokes `_dispatch_agents()` to forward the standalone query to one or more specialists. Each specialist then executes its own `answer()` pipeline: it applies domain-specific metadata filters, searches the vector store for semantically relevant passages, reranks the retrieved candidates, and selects the strongest evidence that fits within the model’s context window. Based on this evidence, the specialist produces a partial answer in which substantive claims are linked to source citations.

3.5.4 Synthesis, Validation, and Memory

When several specialists are involved, their outputs are processed by `_aggregate()`, which removes unnecessary repetition and combines complementary findings into a coherent final response while preserving citations. If only one usable specialist answer is returned, it passes through directly, avoiding an additional generation step that could introduce information loss. If the available documents do not provide sufficient evidence, the system returns a qualified response rather than presenting an unsupported conclusion.

Before delivery, `check_grounding()` verifies that the main claims are supported by the retrieved passages and that the citations are consistent with their sources. Finally, the request and response are stored in conversational memory, allowing later interactions to resolve references, preserve context, and improve routing decisions.

Chapter 4

Evaluation Metrics and Conclusion

4.1 Evaluation Methodology

The four RAG implementations were evaluated using a common procedure based on the Ragas framework. Five complementary metrics were selected to assess the quality of retrieved information and generated answers: **Context precision**, **context recall**, **faithfulness**, **answer relevancy**, and **answer correctness**. All systems were evaluated using the same DeepSeek judge model, configuration, and BGE-M3 embedding model to maintain consistent scoring conditions.

The evaluation dataset consisted of questions paired with reference answers. For each question, the corresponding RAG system generated a response and retrieved supporting document passages. The question, reference answer, generated response, and retrieved passages were stored in an evaluation record. Reference answers were used only during scoring and were not provided to the systems when generating their responses.

Each question was processed in a new session, with long-term memory disabled, to prevent information from previous evaluation cases from influencing subsequent answers. The experiment therefore assessed independent question answering rather than multi-turn conversational performance.

Data collection and scoring were performed as separate stages. This allowed previously collected responses and passages to be rescored without rerunning the RAG pipelines. The evaluator was configured with temperature zero to reduce variability, although LLM-based judgments are not guaranteed to be fully deterministic.

4.2 Evaluation Metrics

Context precision. Context precision measures whether retrieved passages judged relevant to the reference answer appear early in the retrieval order. The implemented metric, `LLMContextPrecisionWithReference`, uses the evaluator to assign a relevance judgment

to each passage. The score is calculated as

$$\text{ContextPrecision} = \frac{\sum_{k=1}^K P(k) v_k}{\sum_{k=1}^K v_k}, \quad P(k) = \frac{\sum_{i=1}^k v_i}{k},$$

where K is the number of retrieved passages and v_k is one when passage k is judged relevant and zero otherwise. The expression assumes at least one relevant passage. A high score indicates that relevant passages are concentrated near the beginning of the saved retrieval order.

For the multi-agent systems, passages are combined in agent registry order, followed by each agent’s internal retrieval order. Consequently, this metric evaluates the saved combined order rather than a globally reranked list of passages.

Context recall. Context recall measures the extent to which the retrieved passages support the information contained in the reference answer. The reference is decomposed into claims, and the evaluator determines whether each claim can be supported by the retrieved context:

$$\text{ContextRecall} = \frac{\text{Number of reference claims supported by the context}}{\text{Total number of reference claims}}.$$

A high score indicates that retrieval provides evidence covering most of the expected answer. This is a measure of reference-claim coverage, rather than document recall over the entire corpus.

Faithfulness. Faithfulness measures whether the claims made in the generated response are supported by the retrieved passages:

$$\text{Faithfulness} = \frac{\text{Number of response claims supported by the context}}{\text{Total number of response claims}}.$$

Unlike context recall, faithfulness examines claims in the generated response rather than claims in the reference answer. It does not directly depend on the reference.

A high score indicates that the answer is grounded in the retrieved evidence. However, it does not establish that the evidence itself is accurate or that the response contains all information needed to answer the question.

Answer relevancy. Answer relevancy assesses how closely the generated response addresses the original question. Ragas generates a set of questions from the response and compares their embeddings with the embedding of the original question using cosine similarity. Responses judged noncommittal are penalized.

This metric does not use the reference answer directly. A high score indicates that the response addresses the requested topic, but does not necessarily establish factual accuracy

or completeness.

Answer correctness. Answer correctness compares the generated response with the reference answer through a combination of factual agreement and semantic similarity:

$$\text{AnswerCorrectness} = 0.75 F + 0.25 S,$$

where F represents the factual comparison component and S represents embedding-based semantic similarity.

The factual component compares claims in the response and reference, accounting for matching claims, additional unmatched claims, and missing reference claims. The semantic component measures similarity in meaning. The score therefore reflects agreement with the selected reference rather than an independent verification of every statement against external sources.

4.3 Influence of Reference-Answer Quality

Reference-answer design is an essential part of the evaluation. Context precision, context recall, and answer correctness depend directly on the reference answer. Faithfulness and answer relevancy do not use it directly and provide complementary perspectives on response quality.

A reference answer defines the expected content and level of detail. A concise reference establishes a narrow target, whereas a more detailed reference may introduce additional facts, conditions, or exceptions. Changing these expectations can alter the scores even when the generated response and retrieved passages remain unchanged.

4.4 Aggregation and Interpretation

Scores were calculated separately for each question and then aggregated using the arithmetic mean of valid numerical results:

$$\overline{m} = \frac{1}{|V_m|} \sum_{i \in V_m} m_i,$$

where V_m is the set of cases with a valid score for metric m .

Generation failures were excluded from scoring. When no passages were retrieved, context precision, context recall, and faithfulness were left unscored, while answer relevancy and answer correctness could still be evaluated. Scoring failures were also excluded from the corresponding metric mean.

The five metrics must be interpreted jointly. High faithfulness with low context recall indicates an answer based on incomplete evidence, while high context recall with low answer correctness suggests relevant evidence was retrieved but poorly utilized. Furthermore, high answer relevancy alone doesn’t ensure correctness. Even combining high faithfulness with high answer relevancy only proves the LLM perfectly processed the provided context it does not guarantee a factually complete or correct final answer if the retrieved information itself is flawed.

4.5 Ragas Results

In this section, we present the evaluation results for the four RAG implementations, highlighting their performance across the five selected metrics.

Each implementation was evaluated using the same set of test questions and ground-truth references. Specifically, we compare the performance differences between single-agent and multi-agent architectures, as well as the impact of using different embedding models contrasting a lightweight model with a higher-parameter alternative.

4.5.1 Performance Comparison

Table 4.1: Evaluation results of the four RAG implementations.

Embedding Model	Architecture	Faithfulness	Context Recall	Answer Rel.	Context Prec.	Answer Corr.
BGE (568M)	Multi-Agent	0.958	0.442	0.604	0.595	0.433
	Single-Agent	0.856	0.225	0.622	0.276	0.551
Nemotron (1B)	Single-Agent	0.971	0.335	0.716	0.630	0.501
	Multi-Agent	0.989	0.371	0.757	0.568	0.518

To ensure a consistent comparison, all four configurations were evaluated using `deepseek-chat` as the Ragas evaluator and `BAAI/bge-m3` as the embedding model. Gemini was initially tested as an alternative evaluator. However, the free-tier request limit was exceeded because Ragas requires multiple API calls per sample. After paid API access was enabled, the project’s monthly spending cap was reached before a single evaluation run could be completed. DeepSeek was therefore retained for all experiments, as its available quota supported the evaluation workload.

General Conclusions

Multi-agent architectures can outperform single-agent systems on complex research tasks by distributing the workload among specialized agents. Each agent can focus on a particular argument, topic, source type, or country, allowing it to develop deeper and more contextually relevant knowledge. This specialization can also improve document retrieval and source organization: instead of producing a single, heterogeneous collection of references, the system can retrieve and organize sources separately for each topic or country.

A coordinating agent can then compare the findings, reconcile inconsistencies, and synthesize the results into a coherent final response.

However, multi-agent systems also introduce additional computational costs and coordination overhead. Their use is therefore most beneficial when a task can be divided into clearly defined subtasks that lend themselves to parallel processing or require specialized analysis.

Although the benefits may not be immediately evident from the quantitative results alone, the integration of **BGE-M3** and **Nemotron** improved the system’s overall performance, particularly in terms of reasoning, instruction following, document retrieval, and the quality of intermediate outputs. To illustrate these qualitative improvements, the following section compares two responses to the same predefined question. The first response was generated by our multi-agent pipeline, which uses **BGE-M3** for document retrieval and **Nemotron** for language generation and reasoning. The second was produced by **Gemini** using its free API tier.

Evaluating both systems on the same question enables a direct comparison of their depth of analysis, relevance, source selection and organization, and overall response quality. The complete responses and a detailed comparison supporting the observations discussed above are available at the following link: [response comparison](#).